Tensors, Gradients and Regression

PyTorch tensors, autograd and the chain rule, what a unit, a layer and a network compute, a straight line fitted by gradient descent by hand and with `nn.Linear`, `nn.MSELoss` and `torch.optim.SGD`, networks written as `nn.Module`, an MLP fitted to a curve that a straight line cannot follow, and a regression on the customer register measured by RMSE and MAE.

01 Tensors
02 Autograd
03 Neural networks
04 Linear regression
05 nn.Module
06 An MLP on a curve
07 Regression on the register

Scope. NumPy arrays, broadcasting and reductions are assumed, and so is the customer register read into arrays with its missing values as NaN.

After reading it you should be able to do the following without looking anything up.

- Create tensors, say which conversions between NumPy and PyTorch copy, and fix a dtype or a device mismatch.
- Compute a derivative with `backward()`, follow the chain rule through a computational graph, say why `.grad` has to be reset, and write one step of gradient descent by hand.
- Say what a unit, a layer and a network compute from a row of numbers.
- Fit a straight line by gradient descent, first with tensors and autograd and then with `nn.Linear`, `nn.MSELoss` and `torch.optim.SGD`.
- Write a network as an `nn.Module`, count its parameters, and say why its layers need an activation between them.
- Train an MLP, and say when it fits better than a straight line and when it does not.
- Report the error of a regression as RMSE and MAE against the baseline of predicting the mean, and keep the prediction and the target the same shape.

Every example assumes `import numpy as np` and `import matplotlib.pyplot as plt`, and the PyTorch examples also `import torch` and `from torch import nn`. The curve examples generate their 200 points in the code that uses them. The register examples use the 9000 customer records read into one float array per numeric column, with missing values as NaN: `tenure`, `score`, `income` and `spend`, the amount each customer spent. `split` holds `"train"` or `"test"` for every row, and `plan` the customer's plan as text. Values in `# ->` comments are rounded to the digits shown.

01

Tensors

A `torch.Tensor` is PyTorch's n-dimensional array: a block of elements of one data type, with a shape. Indexing, slicing, broadcasting, reductions and `@` follow the same rules as on a NumPy array. The differences are the default dtype, the names of two arguments, the device the data is stored on, and the recording of operations for differentiation.

```
t = torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])   # from a nested list, like np.array
t.shape                 # -> torch.Size([2, 3])
t.dtype                 # -> torch.float32

torch.zeros(2, 3)       # (2, 3) of 0.0
torch.ones(2, 3)        # (2, 3) of 1.0
torch.arange(6)         # -> tensor([0, 1, 2, 3, 4, 5])
torch.rand(2, 3)        # (2, 3), uniform on [0, 1)
```

`torch.Size` is a subclass of `tuple`, so `t.shape[0]` and `len(t.shape)` work as they do on a NumPy shape. The creation functions take the shape as separate integers, and also accept a tuple.

DTYPE

```
a = np.array([1.5, 2.5])            # (2,) float64, NumPy's default

torch.tensor([1.5, 2.5]).dtype      # -> torch.float32   PyTorch's default
torch.tensor(a).dtype               # -> torch.float64   the array's dtype is kept
torch.tensor(a, dtype=torch.float32).dtype      # -> torch.float32

layer = torch.nn.Linear(2, 1)       # a weighted sum of 2 inputs, with float32 weights
layer(torch.tensor(a))
# RuntimeError: mat1 and mat2 must have the same dtype, but got Double and Float
```

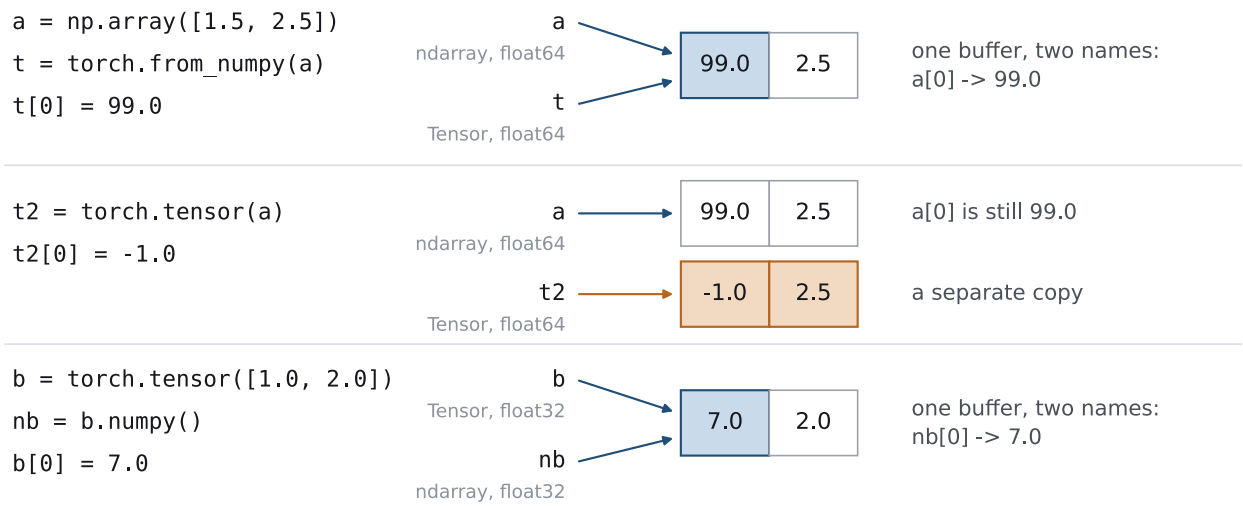
NumPy creates float64 and PyTorch creates float32. A tensor made from a NumPy array keeps the array's float64, and multiplying it with the float32 weights of a layer raises the error above, in which `Double` is PyTorch's name for float64 and `Float` for float32. Passing `dtype=torch.float32` when the tensor is created avoids it, and `t.float()` converts an existing tensor. float32 takes half the memory of float64.

COPIES AND SHARED MEMORY

```
a = np.array([1.5, 2.5])
t = torch.from_numpy(a)             # shares a's memory
t[0] = 99.0
a[0]                                # -> 99.0   the array changed too

t2 = torch.tensor(a)                # a copy
t2[0] = -1.0
a[0]                                # -> 99.0   unchanged

b = torch.tensor([1.0, 2.0])
nb = b.numpy()                      # shares b's memory
b[0] = 7.0
nb[0]                               # -> 7.0
```



torch.from_numpy(a) and b.numpy() give a second name for the same buffer, so the write shows through both names.
torch.tensor(a) copies the values into a buffer of its own, and writing to t2 leaves a unchanged.

`torch.from_numpy` and `.numpy()` share memory with the object they are called on, the way a NumPy view shares memory with its base array, so a write through one shows in the other. `torch.tensor` always copies. Sharing avoids a copy, and leads to errors when one side is modified while the other was expected to stay unchanged. `torch.from_numpy` also keeps the array's dtype, so a float64 array still has to be converted.

REDUCTIONS AND THE MATRIX PRODUCT

```
x = torch.zeros(5, 4)
x[:, 0].shape                       # -> torch.Size([5])   an integer index removes an axis
x.sum(dim=1).shape                  # -> torch.Size([5])
x.mean(dim=0, keepdim=True).shape   # -> torch.Size([1, 4])
x.T.shape                           # -> torch.Size([4, 5])

X = torch.tensor([[47.0, 20.0], [57.0, 19.0], [36.0, 9.0]])    # 3 rows, 2 features
w = torch.tensor([0.5, -1.0])                                    # one weight per feature
X @ w                               # -> tensor([3.5000, 9.5000, 9.0000])   one per row
```

PyTorch's names for NumPy's `axis` and `keepdims` are `dim` and `keepdim`. The reductions also accept the NumPy names, but the PyTorch documentation and most PyTorch code use `dim` and `keepdim`. `X @ w` multiplies every row of `X` by the weights and adds the products, which is the weighted sum a layer of a network computes for every row at once.

DEVICE

```
torch.cuda.is_available()           # -> True if PyTorch can use a GPU
device = "cuda" if torch.cuda.is_available() else "cpu"

t = torch.tensor([1.0, 2.0])        # created in main memory
g = t.to(device)                    # a copy in GPU memory
g.device                            # -> device(type='cuda', index=0)

g + t
# RuntimeError: Expected all tensors to be on the same device, but found at least
# two devices, cuda:0 and cpu!

g.cpu().numpy()                     # -> array([1., 2.], dtype=float32)
```

Every tensor is stored on a device: `cpu` is main memory, `cuda` is the memory of a GPU. `.to(device)` returns a copy on that device, or the tensor itself if it is already there. An operation between tensors on different devices raises the error above, and `.numpy()` works only on a tensor in main memory, so a result computed on the

GPU goes through `.cpu()` first. Choosing the device once, as in the second line, lets the same code run with or without a GPU. In Colab a GPU is selected under Runtime, then Change runtime type.

02

Autograd

Autograd is PyTorch's automatic differentiation. Every operation on a tensor created with `requires_grad=True` is recorded, and `backward()` uses that record to compute the derivative of a result with respect to the tensor.

```
x = torch.tensor(5.0, requires_grad=True)

y = 3 * x ** 2 + 4 * x + 2          # -> tensor(97., grad_fn=<AddBackward0>)
y.backward()                        # dy/dx = 6x + 4, which is 34 at x = 5
x.grad                              # -> tensor(34.)
```

`requires_grad=True` marks `x` as a tensor to differentiate with respect to. `backward()` computes the derivative of `y` with respect to every such tensor that `y` was computed from, and stores each derivative in that tensor's `.grad`. Called without arguments, `backward()` needs `y` to hold a single number, which a loss always does; on a tensor with several elements it raises `RuntimeError: grad can be implicitly created only for scalar outputs`.

THE COMPUTATIONAL GRAPH

```
a = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(3.0, requires_grad=True)
c = torch.tensor(5.0, requires_grad=True)
d = torch.tensor(10.0, requires_grad=True)

u = a * b                           # 6
v = c * torch.log10(d)              # 5
e = u + v                           # -> tensor(11., grad_fn=<AddBackward0>)

a.is_leaf, u.is_leaf, e.is_leaf     # -> (True, False, False)
a.grad_fn, u.grad_fn                # -> (None, <MulBackward0 object at 0x...>)
```

Autograd records the computation as a directed acyclic graph. The tensors created directly, here `a`, `b`, `c` and `d`, are its **leaves**, every operation is a node, and the final result `e` is its root. A tensor produced by an operation holds the node that produced it in `grad_fn`, named after the operation: `MulBackward0` for a product, `AddBackward0` for a sum. A leaf has no `grad_fn`.

```
e.grad_fn.next_functions   # -> ((<MulBackward0 ...>, 0), (<MulBackward0 ...>, 0))
v.grad_fn.next_functions   # -> ((<AccumulateGrad ...>, 0), (<Log10Backward0 ...>, 0))
u.grad_fn._saved_self, u.grad_fn._saved_other    # -> a and b, kept from the forward pass
```

`next_functions` holds, for each input of a node, the node that produced that input, so the recorded graph can be followed from `e` back to the leaves. `c * torch.log10(d)` is two operations, so a `Log10Backward0` node sits between `d` and the second `MulBackward0`. An `AccumulateGrad` node stands for a leaf: it adds the gradient that reaches the leaf into its `.grad`. Each node also keeps the values its derivative needs from the forward pass: the `MulBackward0` of `u` holds `a` and `b`.

```
e.backward()
a.grad, b.grad          # -> (tensor(3.), tensor(2.))   de/da = b, de/db = a
c.grad                  # -> tensor(1.)                 de/dc = log10(d) = 1
d.grad                  # -> tensor(0.2171)             de/dd = c / (d ln 10)
u.grad                  # -> None, with a warning       u is not a leaf
```

THE CHAIN RULE

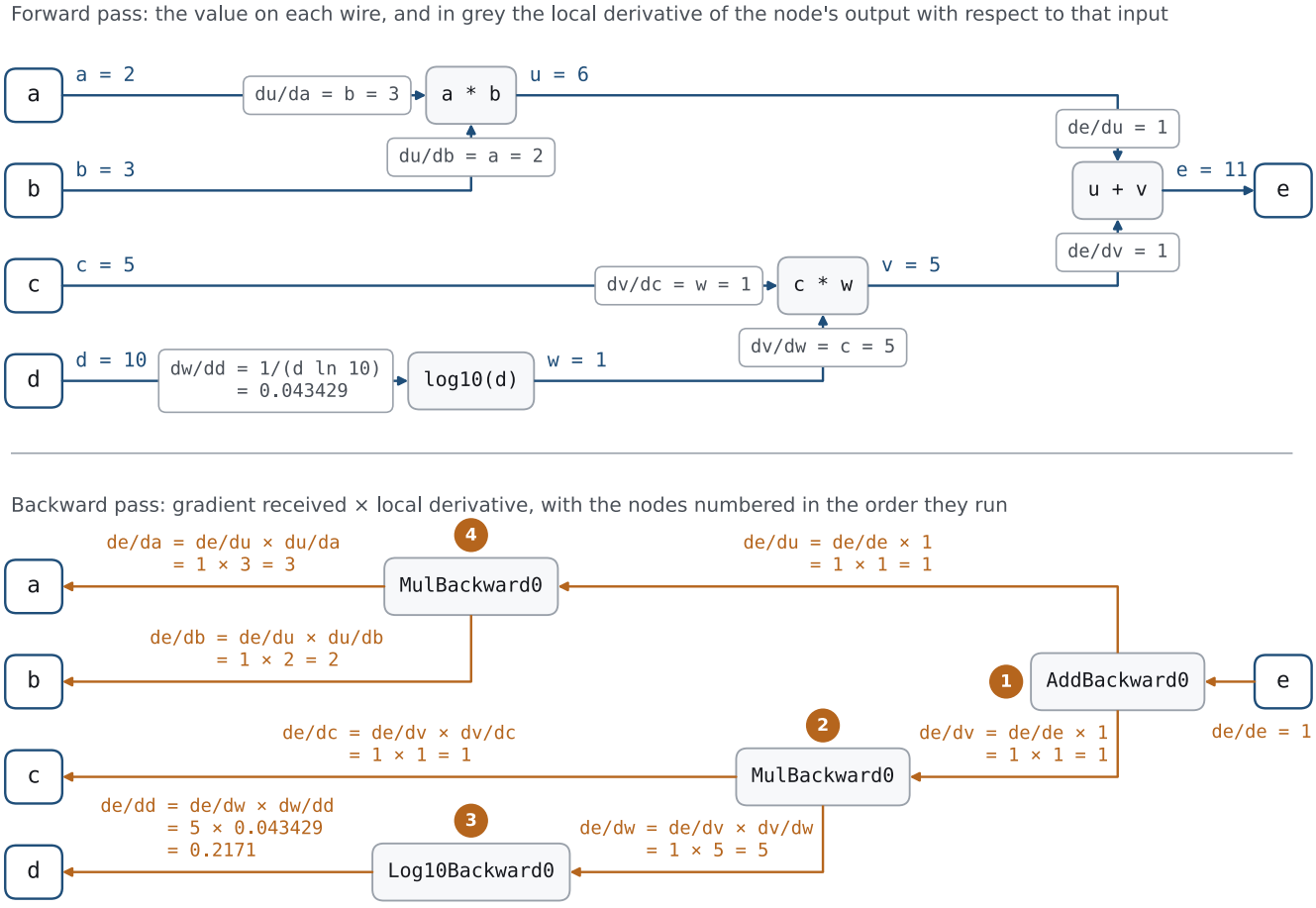
The gradients above follow from the chain rule. Writing `w` for `log10(d)`, the forward pass is four operations: `u = a * b`, `w = log10(d)`, `v = c * w` and `e = u + v`. `e` depends on `d` only along the path d → w → v → e, and the chain rule gives the derivative of `e` with respect to `d` as the product of one derivative per operation on that path: de/dd = de/dv × dv/dw × dw/dd.

Each factor is a **local derivative**: the derivative of one operation's output with respect to one of its inputs, with the other inputs held at their values from the forward pass. A node computes its local derivatives from its own inputs alone. With `a = 2`, `b = 3`, `c = 5` and `d = 10`, the forward pass gives u = 6, w = 1 and v = 5. Holding one input of an operation fixed leaves a function of the other input, and the local derivative is the derivative of that function:

OPERATION	HELD FIXED	FUNCTION OF ONE INPUT	LOCAL DERIVATIVE
`e = u + v`	v = 5	e = u + 5	de/du = 1
	u = 6	e = 6 + v	de/dv = 1
`u = a * b`	b = 3	u = 3 × a	du/da = b = 3
	a = 2	u = 2 × b	du/db = a = 2
`v = c * w`	w = 1	v = 1 × c	dv/dc = w = 1
	c = 5	v = 5 × w	dv/dw = c = 5
`w = log10(d)`	no other input	w = ln(d) / ln 10	dw/dd = 1 / (d ln 10) = 0.043429

Adding an amount to one input of a sum adds the same amount to the sum, so both local derivatives of `u + v` are 1. A product with one factor held fixed is that factor times the other input, and the derivative of k × x with respect to x is k, so the local derivative of a product with respect to one factor is the other factor: with `c` held at 5, v = 5 × w, and dv/dw = c = 5. For the logarithm, log10(d) = ln(d) / ln 10 and the derivative of ln(d) is 1 / d, so dw/dd = 1 / (d ln 10) = 1 / (10 × 2.302585) = 0.043429. The product of the three local derivatives along the path from `d` to `e` is the value of `d.grad` above:

$$
\begin{aligned}
\frac{de}{dd} &= \frac{de}{dv} \times \frac{dv}{dw} \times \frac{dw}{dd} && \text{the chain rule along d} \to \text{w} \to \text{v} \to \text{e} \\
&= 1 \times c \times 1 \,/\, (d \ln 10) && \text{the local derivatives from the table} \\
&= 1 \times 5 \times 0.043429 && c = 5 \text{ and } d = 10 \\
&= 0.2171
\end{aligned}
$$



Top: the forward pass, with the value on every wire and, in grey, the local derivative of each node's output with respect to that input. Bottom: the backward pass, with the nodes numbered in the order they run. Every gradient is the gradient its node receives times a local derivative from the top panel, for example de/dw = de/dv × dv/dw = 1 × 5 = 5.

`backward()` computes every gradient in one pass from the root to the leaves. It starts from de/de = 1 at `e`, the derivative of `e` with respect to itself, and runs a node only after every node that uses its output. A node receives the gradient of `e` with respect to its output, multiplies it by its local derivative for each input, and passes each product on to that input. In this graph PyTorch runs the nodes in this order:

1. `AddBackward0` receives de/de = 1. Both of its local derivatives are 1, so it passes de/du = de/de × 1 = 1 × 1 = 1 and de/dv = de/de × 1 = 1 × 1 = 1.
2. The `MulBackward0` of `v` receives de/dv = 1 and passes de/dc = de/dv × dv/dc = 1 × 1 = 1 and de/dw = de/dv × dv/dw = 1 × 5 = 5.
3. `Log10Backward0` receives de/dw = 5 and passes de/dd = de/dw × dw/dd = 5 × 0.043429 = 0.2171.
4. The `MulBackward0` of `u` receives de/du = 1 and passes de/da = de/du × du/da = 1 × 3 = 3 and de/db = de/du × du/db = 1 × 2 = 2.

At the root the received gradient is de/de = 1, so the gradients that `AddBackward0` passes equal its local derivatives. Steps 2 and 3 evaluate the product de/dv × dv/dw × dw/dd from the left: step 2 computes de/dw = de/dv × dv/dw = 5, and step 3 multiplies it by dw/dd. Each gradient is computed once and reused by the nodes between it and the leaves, without returning to `e`. The same pass, written out in plain Python, gives the four values of `.grad`:

```
import math

a, b, c, d = 2.0, 3.0, 5.0, 10.0       # the same values, as plain floats
w = math.log10(d)                      # 1.0, the forward value of log10(d)

de_de = 1.0                            # the start: e with respect to itself
de_du = de_de * 1.0                    # AddBackward0: de/du = 1
de_dv = de_de * 1.0                    #               de/dv = 1
de_dc = de_dv * w                      # MulBackward0 of v: dv/dc = w
de_dw = de_dv * c                      #                    dv/dw = c
de_dd = de_dw / (d * math.log(10))     # Log10Backward0: dw/dd = 1 / (d ln 10)
de_da = de_du * b                      # MulBackward0 of u: du/da = b
de_db = de_du * a                      #                    du/db = a

de_da, de_db, de_dc, round(de_dd, 4)   # -> (3.0, 2.0, 1.0, 0.2171)
```

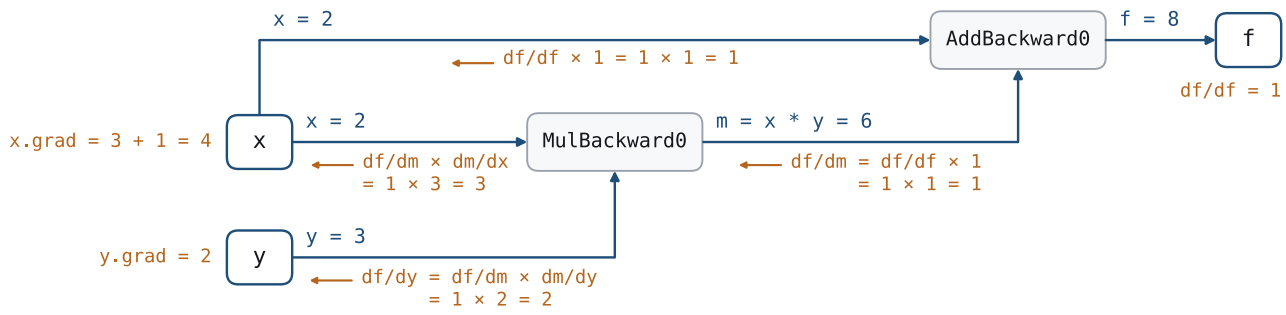
Only the leaves keep their gradient in `.grad`. The gradients of intermediate results are computed in the same pass and then discarded. `retain_grad()` keeps them, and they are the values passed between the nodes above:

```
a, b, c, d = [torch.tensor(x, requires_grad=True) for x in (2.0, 3.0, 5.0, 10.0)]
u = a * b
w = torch.log10(d)
v = c * w
e = u + v
for t in (u, v, w):
    t.retain_grad()                    # keep the gradient of an intermediate result
e.backward()
u.grad, v.grad, w.grad                 # -> (tensor(1.), tensor(1.), tensor(5.))
```

GRADIENTS ALONG SEVERAL PATHS

When a tensor is used more than once, a gradient reaches it along each path, and the gradients are added.

```
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)
f = x * y + x                          # x is used twice
f.backward()
x.grad, y.grad                         # -> (tensor(4.), tensor(2.))   y + 1 and x
```



f = x * y + x with x = 2 and y = 3, writing m for x * y. x reaches f through the product, df/dm × dm/dx = 1 × 3 = 3, and directly through the sum, df/df × 1 = 1 × 1 = 1; the two are added, so x.grad = 4.

Writing `m` for `x * y`, `x` reaches `f` along two paths: through the product, whose local derivative with respect to `x` is dm/dx = y = 3, and directly through the sum, whose local derivative with respect to `x` is 1. The gradient along the first path is df/dm × dm/dx = 1 × 3 = 3, and along the second df/df × 1 = 1 × 1 = 1. The two are added before the `AccumulateGrad` node of `x` runs, so it receives 4 once, and df/dx = y + 1.

GRADIENT ACCUMULATION

```
x = torch.tensor(5.0, requires_grad=True)
(3 * x ** 2 + 4 * x + 2).backward()
(3 * x ** 2 + 4 * x + 2).backward()     # a second pass on a newly built graph
x.grad                                  # -> tensor(68.)   34 added to 34

x.grad.zero_()
(3 * x ** 2 + 4 * x + 2).backward()
x.grad                                  # -> tensor(34.)
```

`backward()` adds into `.grad` instead of overwriting it, so that the gradients of several losses can be summed into one tensor. A loop that calls `backward()` once per step therefore resets `.grad` between calls. `zero_()` sets every element to zero in place; the trailing underscore marks every PyTorch method that modifies the tensor it is called on. `backward()` also frees the graph, which is why each pass above computes the expression again: a second `backward()` on the same result raises a `RuntimeError`.

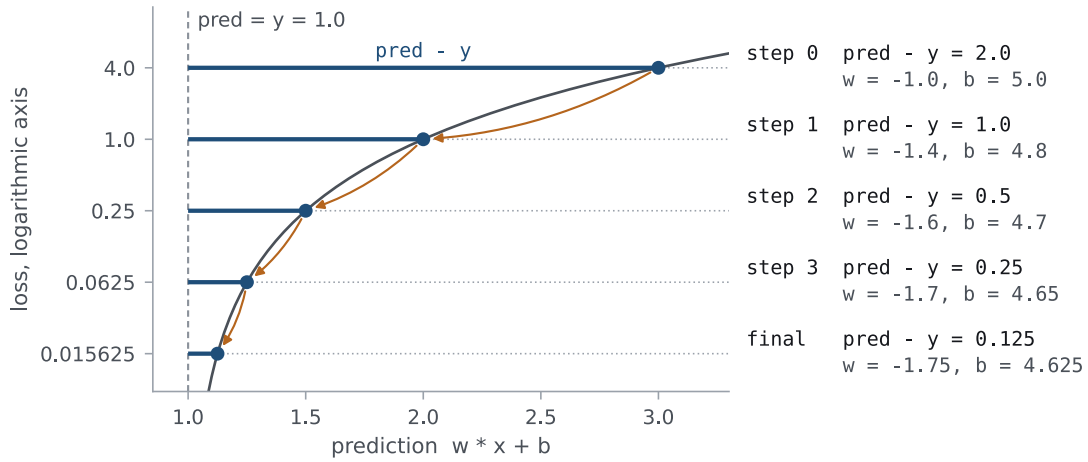
ONE STEP OF GRADIENT DESCENT

```
x, y = 2.0, 1.0                                 # one input and its target
w = torch.tensor(-1.0, requires_grad=True)      # weight
b = torch.tensor(5.0, requires_grad=True)       # bias
lr = 0.05                                       # learning rate

for step in range(4):
    pred = w * x + b
    loss = (pred - y) ** 2
    loss.backward()
    with torch.no_grad():
        w -= lr * w.grad
        b -= lr * b.grad
    w.grad.zero_()
    b.grad.zero_()
```

STEP	PREDICTION	LOSS	`w.grad`	`b.grad`
0	3.0	4.0	8.0	4.0
1	2.0	1.0	4.0	2.0
2	1.5	0.25	2.0	1.0
3	1.25	0.0625	1.0	0.5

Gradient descent moves every parameter a small step against its gradient: `w - lr * w.grad` replaces `w`. The gradient points in the direction in which the loss increases fastest, so a small enough step the other way decreases it. Here `dL/dw = 2 * (pred - y) * x` and `dL/db = 2 * (pred - y)`. The first step moves `w` from -1.0 to -1.4 and `b` from 5.0 to 4.8, and with this learning rate every step halves the error `pred - y`, so the loss falls to a quarter each time. After the four steps `w` is -1.75 and `b` is 4.625.



The loss depends on w and b only through the prediction w * x + b. Each step moves the prediction half of the remaining way to y = 1.0, so the error pred - y halves from 2.0 to 0.125 and the loss falls to a quarter, the same distance every time on the logarithmic axis.

TORCH.NO_GRAD

The update runs inside `torch.no_grad()`, which stops the recording, because the update is not part of the computation being differentiated. Outside it, the in-place `w -= lr * w.grad` raises `RuntimeError: a leaf Variable that requires grad is being used in an in-place operation.` Writing `w = w - lr * w.grad` instead raises no error, but it creates a new tensor computed from `w`, which is not a leaf, so the next `backward()` leaves its `.grad` as `None`.

```
with torch.no_grad():
    pred = w * 3.0 + b              # pred.requires_grad -> False
```

The same context manager wraps any computation whose result will not be differentiated, such as predictions on test data. Nothing is recorded, which saves the memory and the time the record takes.

DETACH AND ITEM

```
w.numpy()
# RuntimeError: Can't call numpy() on Tensor that requires grad.
# Use tensor.detach().numpy() instead.
w.detach().numpy()                  # -> array(-1.75, dtype=float32)
w.item()                            # -> -1.75   a Python float
```

`detach()` returns a tensor with the same data that is not part of any graph, which `.numpy()` accepts. `.item()` returns the value of a one-element tensor as a Python number, which is how a loss is usually printed or stored.

`torch.optim.SGD` performs this same update in its `step()` method, for every parameter it was given, and its `zero_grad()` clears every `.grad`, setting it to `None` rather than to zero.

03

Neural networks

A neural network is a function from one row of numbers to a prediction, built from weighted sums of its inputs.

ONE UNIT

```
x = np.array([57.0, 18581.51, 19.0, 45.8])   # age, income, tenure_months, score
w = np.array([0.02, 0.0001, -0.05, 0.03])    # one weight per input
b = -1.0                                     # the bias

x * w                   # -> [ 1.14   1.858 -0.95   1.374]   one product per input
z = x @ w + b
z                       # -> 2.422   the weighted sum plus the bias
np.maximum(z, 0.0)      # -> 2.422   ReLU: a negative sum would become 0
```

A **unit**, also called a neuron, multiplies each input by its own weight, adds the products and a **bias**, and passes the result through an **activation function**. The most common one is ReLU, `max(z, 0)`, which keeps a positive sum and replaces a negative one with 0. The weights and the bias are the unit's **parameters**: numbers that are set by training rather than written by hand. Here they were written by hand, and for income's product to be about as large as the others, income's weight had to be 200 times smaller than age's.

A LAYER

```
W = np.array([[0.02, 0.0001, -0.05, 0.03],      # (3, 4): one row of weights per unit
              [-0.01, -0.0001, 0.04, 0.01],
              [0.03, 0.00002, 0.02, -0.04]])
bias = np.array([-1.0, 0.5, 0.2])               # one bias per unit

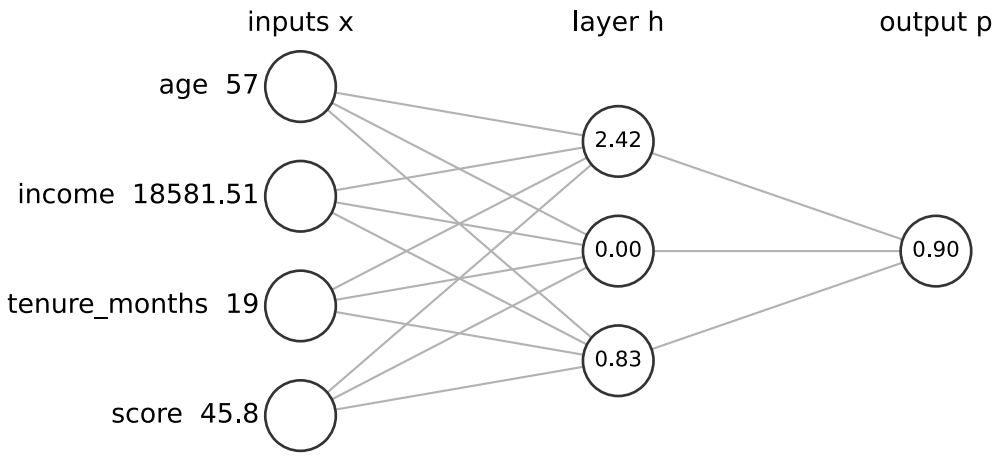
W @ x + bias            # -> [ 2.422 -0.71   0.83 ]   (3, 4) @ (4,) + (3,) -> (3,)
h = np.maximum(W @ x + bias, 0.0)
h                       # -> [2.422 0.    0.83 ]     the second sum was negative
```

A **layer** is several units that read the same inputs. Its weights form a matrix with one row per unit, so one matrix product computes every unit at once, and the bias adds one number per unit. The output `h` is a new vector with one number per unit, and the next layer reads `h` as its input. A network is a stack of layers, and the layers between the input and the output are called **hidden layers**.

THE OUTPUT

```
v = np.array([0.8, -0.6, 0.5])          # the output unit: one weight per unit of the layer
c = -0.2                                # and its bias
p = 1 / (1 + np.exp(-(v @ h + c)))
p                                       # -> 0.896
```

For a yes-or-no label, such as whether a customer leaves, the last layer is a single unit whose sum goes through the **sigmoid**, `1 / (1 + exp(-z))`, which maps any number to a value between 0 and 1 that is read as a probability. These weights were chosen by hand, so 0.896 is not a prediction about this customer; training sets the weights. For an amount, such as how much a customer spends, the last unit has no activation, and its weighted sum is the prediction.



The network with the values for the customer above: the inputs x, the layer outputs h = max(W @ x + bias, 0), and the output p after the sigmoid, 0.896. Every line is one weight: 12 between the inputs and the layer, the rows of W, and 3 between the layer and the output, the entries of v.

```
X = np.array([[57.0, 18581.51, 19.0, 45.8],     # (2, 4): two customers, one per row
              [48.0, 35917.79, 15.0, 58.6]])
np.maximum(X @ W.T + bias, 0.0)                 # (2, 4) @ (4, 3) + (3,) -> (2, 3)
# [[2.422 0.    0.83 ]
#  [4.56  0.    0.314]]
```

A table goes through the same way. With one customer per row of `X`, `X @ W.T` computes every unit for every customer in one product, and the bias broadcasts across the rows. This is why the input of a network is a matrix: one row per example, one column per input.

TRAINING

The weights start as random numbers. Training passes rows whose label is known through the network, measures how far its predictions are from the labels with a **loss**, a single number averaged over the rows, and moves every weight a small step in the direction that lowers the loss. That direction comes from the **gradient**, the derivative of the loss with respect to each weight. The derivative of a weighted sum with respect to one weight is the input that weight multiplies, so the step a weight takes is proportional to its input: for the customer above, in the same unit, the step for income's weight is 406 times the step for score's.

04

Linear regression

A straight line, `w * x + b`, fitted to a set of points by gradient descent on the mean squared error between its predictions and the targets. The points lie on a curve with noise added, and are generated by the code that uses them.

```
rng = np.random.default_rng(0)
x = rng.uniform(-3, 3, 200)
y = np.sin(1.5 * x) + 0.3 * x + rng.normal(0, 0.15, 200)   # a curve plus noise

X = torch.tensor(x, dtype=torch.float32).reshape(-1, 1)    # (200,) -> (200, 1)
Y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
X.shape                 # -> torch.Size([200, 1])   one row per point, one column per input
```

The inputs of a model form a matrix with one row per example and one column per input, here a single column. `reshape(-1, 1)` gives one column and as many rows as the data fills. The targets get the same shape, `(200, 1)`, which the loss below requires, and both are float32, the dtype of the parameters of a PyTorch layer.

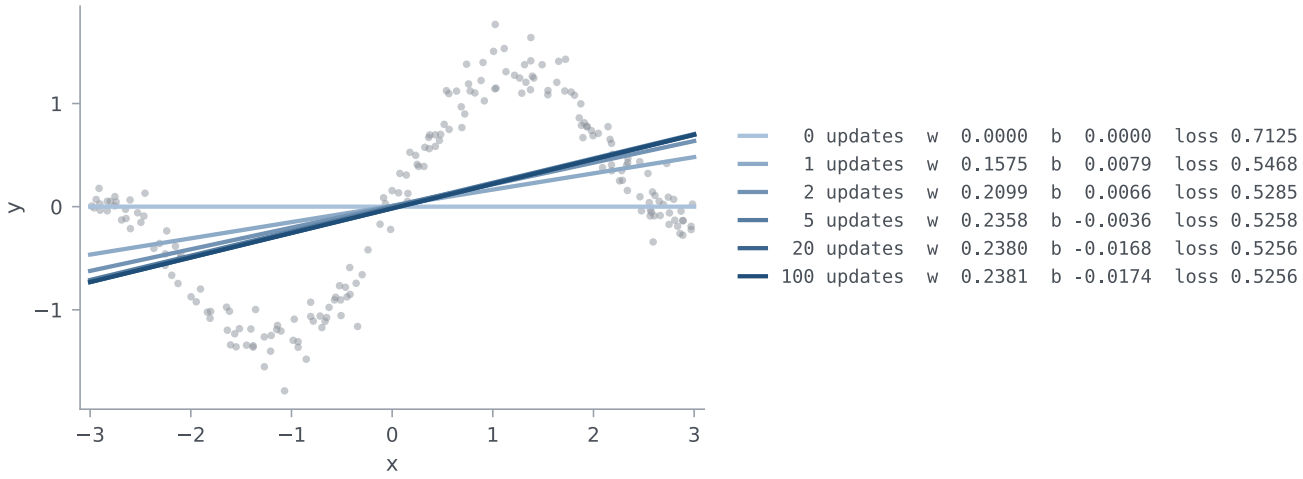
THE LINE BY HAND

```
w = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)

for step in range(100):
    pred = X * w + b                        # (200, 1) * (1,) + (1,) -> (200, 1)
    loss = ((pred - Y) ** 2).mean()         # the mean squared error
    loss.backward()
    with torch.no_grad():
        w -= 0.1 * w.grad
        b -= 0.1 * b.grad
    w.grad.zero_()
    b.grad.zero_()

round(w.item(), 4), round(b.item(), 4)      # -> (0.2381, -0.0174)
round(loss.item(), 4)                       # -> 0.5256
np.polyfit(x, y, 1).round(4)                # -> [ 0.2381 -0.0174]   solved directly
```

The loop repeats one step of gradient descent 100 times over all 200 points. The loss is the **mean squared error**, the mean of `(pred - Y) ** 2` over the rows, so every point contributes to both gradients. After 100 steps the weight is 0.2381 and the bias -0.0174, the **least-squares line**, the line with the smallest mean squared error: `np.polyfit(x, y, 1)` computes it directly and returns the same two numbers.



The line after 0, 1, 2, 5, 20 and 100 updates of the loop above, darker as the updates go on. The first update removes most of the loss; after 20 the line is within a thousandth of the least-squares line, and its loss no longer changes in the fourth decimal.

NN.LINEAR, NN.MSELOSS AND TORCH.OPTIM.SGD

```
torch.manual_seed(0)
line = nn.Linear(1, 1)                      # weight (1, 1), bias (1,)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(line.parameters(), lr=0.1)

for step in range(100):
    loss = loss_fn(line(X), Y)
    opt.zero_grad()                         # every .grad to None
    loss.backward()
    opt.step()                              # every parameter minus lr times its gradient

round(line.weight.item(), 4), round(line.bias.item(), 4)   # -> (0.2381, -0.0174)
```

The same model and the same loop, written with three PyTorch objects. `nn.Linear(1, 1)` holds a weight and a bias as tensors with `requires_grad=True`, starting from random values, and `line(X)` computes `X @`

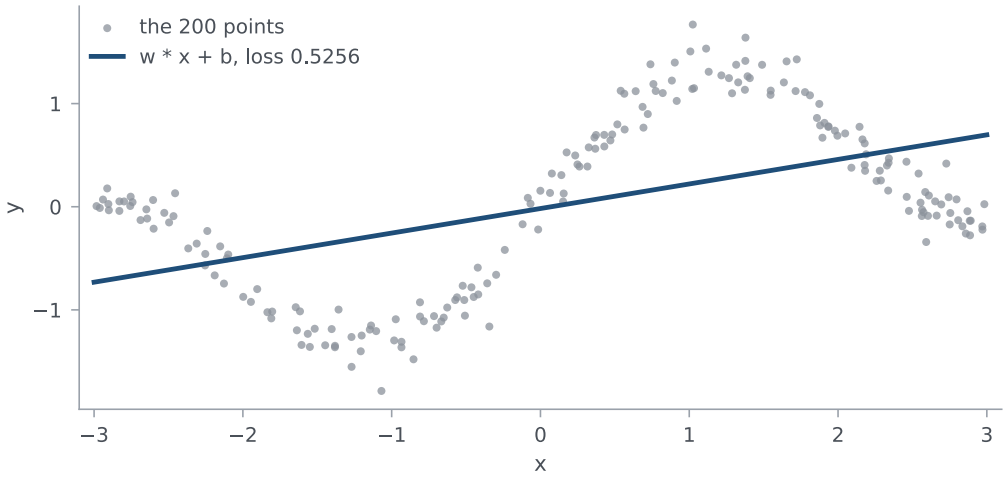
`line.weight.T + line.bias`. `nn.MSELoss()` returns a function that computes the mean squared error of its two arguments. `torch.optim.SGD` holds the parameters it is given and the learning rate: `opt.zero_grad()` sets the `.grad` of every parameter to `None`, and `opt.step()` subtracts `lr` times its gradient from every parameter, the update the loop above writes out. Inside the loop the order is fixed: compute the loss, clear the old gradients, compute the new ones, update. `torch.manual_seed(0)` fixes the random starting values, so a run can be repeated.

	start of the second step	loss = loss_fn(line(X), Y)	opt.zero_grad()	loss.backward()	opt.step()
line.weight	0.1295	0.1295	0.1295	0.1295	0.1801
line.bias	0.4374	0.4374	0.4374	0.4374	0.3516
weight.grad	-1.3701	-1.3701	None	-0.5059	-0.5059
bias.grad	0.9908	0.9908	None	0.8578	0.8578
loss		0.7481	0.7481	0.7481	0.7481

blue: a changed value orange: a changed gradient hatched: not computed yet

The second step of the loop above, one column per statement. opt.zero_grad() sets the gradients left by the first step to None, loss.backward() computes the new ones, and opt.step() subtracts 0.1 times each from its parameter: 0.1295 - 0.1 × -0.5059 = 0.1801.

`nn.Linear` started from a weight of -0.0075 and a bias of 0.5364, the loop above from zeros, and both end at the same line, because the mean squared error of a straight line has a single minimum and 100 steps at this learning rate reach it.



The 200 points and the least-squares line, weight 0.2381 and bias -0.0174. Its loss is 0.5256, against 0.7109 for the flat line at the mean of y: the line follows the overall rise of the points and none of their bends.

05

nn.Module

A network is an object that holds its parameters and applies them in a method, written as an ordinary class: state in `__init__`, behaviour in `forward`.

```
class Net(nn.Module):
    def __init__(self, n_in, n_hidden):
        super().__init__()                  # required, before anything else
        self.stack = nn.Sequential(
            nn.Linear(n_in, n_hidden), nn.ReLU(),
            nn.Linear(n_hidden, 1))

    def forward(self, x):                   # (n, n_in) -> (n, 1)
        return self.stack(x)

torch.manual_seed(0)
net = Net(1, 16)
net(X).shape            # -> torch.Size([200, 1])   calls forward and the hooks around it
```

`super().__init__()` runs `nn.Module`'s setup, which creates the dictionaries that hold the module's layers and parameters; without it, assigning a layer to `self` raises `AttributeError: cannot assign module before Module.__init__() call`. Layers are assigned to `self`, which registers them with the module. `forward` defines the computation. The object is called, `net(X)`, rather than `net.forward(X)`, because `nn.Module.__call__` runs the registered hooks before and after `forward`.

NN.LINEAR AND NN.RELU

```
lin = nn.Linear(1, 16)
lin.weight.shape                # -> torch.Size([16, 1])   (out, in), not (in, out)
lin.bias.shape                  # -> torch.Size([16])
lin(torch.zeros(5, 1)).shape    # -> torch.Size([5, 16])   the last axis is transformed
```

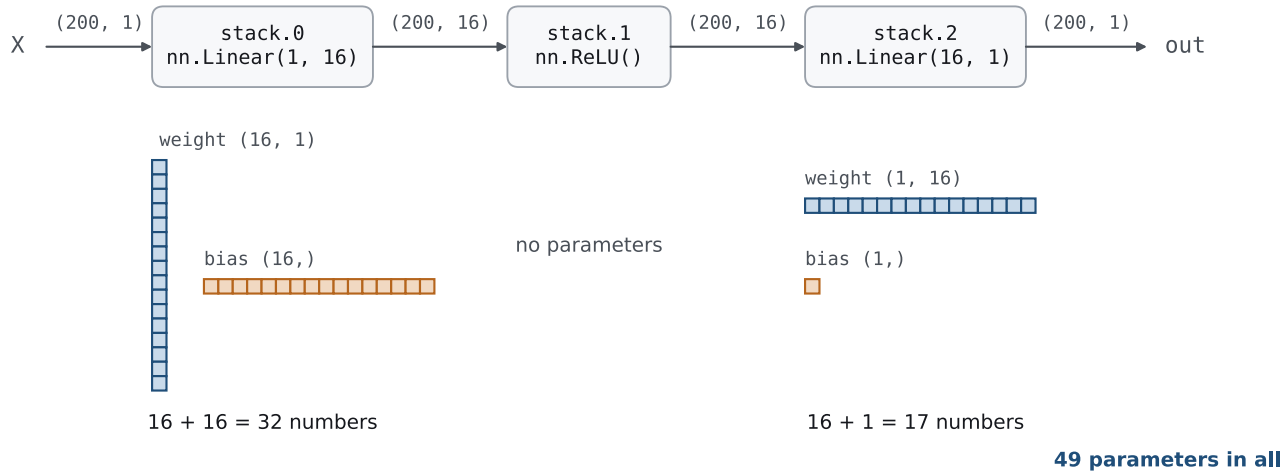
`nn.Linear(a, b)` is a layer of `b` units reading `a` inputs: it computes `x @ W.T + bias`, so every row of the input becomes a row of `b` numbers. `nn.ReLU()` applies `max(x, 0)` elementwise and has no parameters. `nn.Sequential` calls what it holds in order, so `Net(1, 16)` is one input, a hidden layer of 16 units with ReLU, and one output unit with no activation, the form a prediction of an amount takes.

PARAMETER COUNT

```
sum(p.numel() for p in net.parameters())       # -> 49

[(name, tuple(p.shape)) for name, p in net.named_parameters()]
# -> [('stack.0.weight', (16, 1)), ('stack.0.bias', (16,)),
#     ('stack.2.weight', (1, 16)), ('stack.2.bias', (1,))]
```

16 by 1 plus 16, then 1 by 16 plus 1: 49 numbers, all of them adjusted by training. `parameters()` is what an optimizer is given. The indices skip 1 because `nn.Sequential` numbers the activation too.



Net(1, 16) applied to X: the shape of what each layer returns, and the parameters each layer holds, one cell per number. Each nn.Linear holds a weight of shape (out, in) and a bias of shape (out,); nn.ReLU holds none.

LAYERS AS ATTRIBUTES

```
class Net2(nn.Module):
    def __init__(self, n_in, n_hidden):
        super().__init__()
        self.hidden = nn.Linear(n_in, n_hidden)
        self.out = nn.Linear(n_hidden, 1)

    def forward(self, x):
        return self.out(torch.relu(self.hidden(x)))   # ReLU, called as a function

[name for name, p in Net2(1, 16).named_parameters()]
# -> ['hidden.weight', 'hidden.bias', 'out.weight', 'out.bias']
```

The same network without `nn.Sequential`. Every layer that has parameters is assigned to an attribute, which registers it, and `forward` states the order explicitly. ReLU has no parameters, so it is called as a function: `torch.relu` computes the same as `nn.ReLU()`, and so does `torch.nn.functional.relu`. The parameter count is the same 49; only the names change. This form is needed as soon as `forward` does more than apply layers in a fixed order, for example when it uses one input in two places.

06

An MLP on a curve

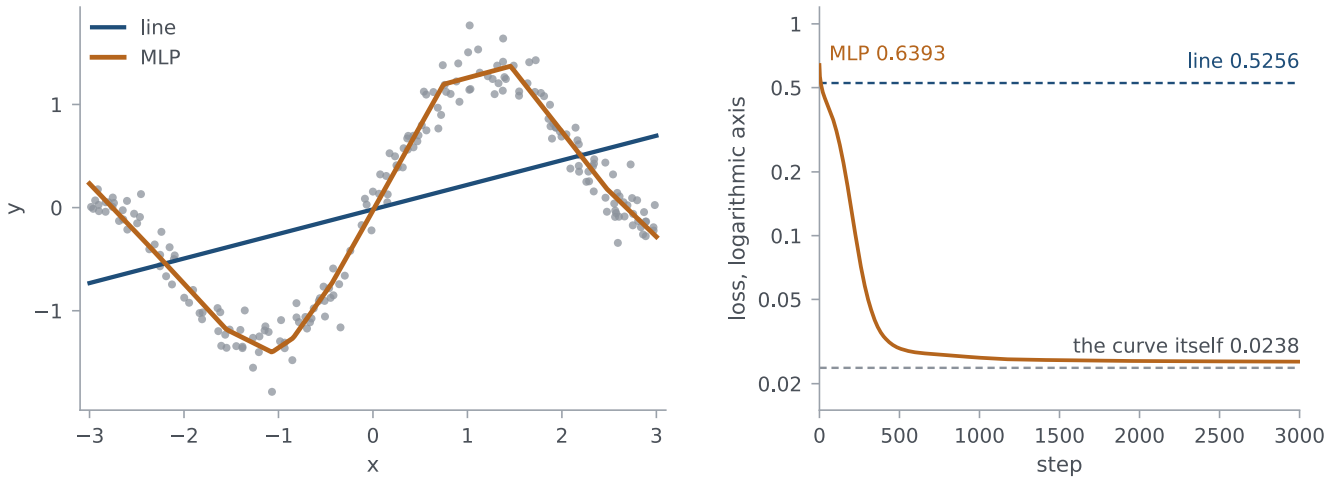
A network with one or more hidden layers between its input and its output is a **multilayer perceptron**, MLP. Trained on the same 200 points with the same loop, `Net(1, 16)` follows the bends that the straight line misses.

```
torch.manual_seed(0)
net = Net(1, 16)
opt = torch.optim.SGD(net.parameters(), lr=0.05)

losses = []
for step in range(3000):
    loss = loss_fn(net(X), Y)
    losses.append(loss.item())
    opt.zero_grad()
    loss.backward()
    opt.step()

round(losses[0], 4), round(losses[-1], 4)      # -> (0.6393, 0.0254)
```

The loss falls from 0.6393 to 0.0254, against 0.5256 for the line. The noise added to the points has a standard deviation of 0.15, so even the curve the points were drawn from has a loss of 0.0238 on them, and the MLP ends 0.0016 above it. Most of the descent happens early: the loss is 0.0293 at step 500, 0.0265 at step 1000 and 0.0256 at step 2000.



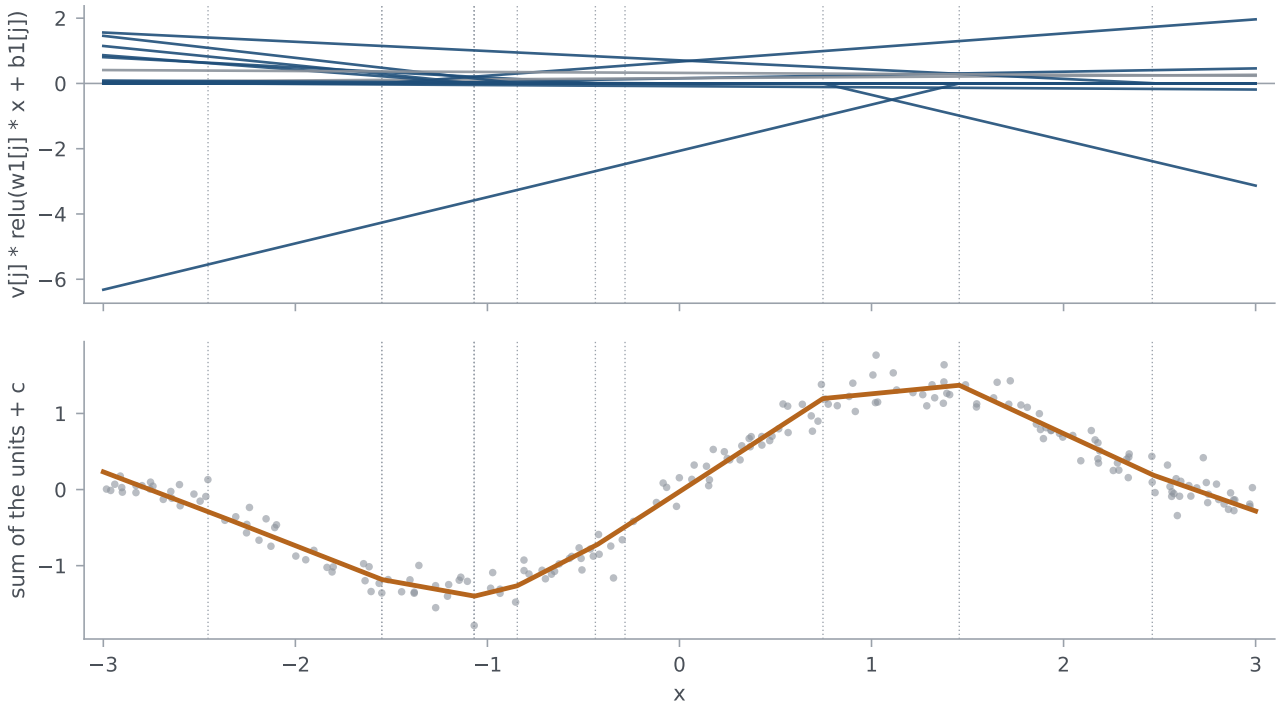
Left: the 200 points with the line and the MLP after training. Right: the MLP's loss at each of its 3000 steps on a logarithmic axis, with the line's loss and the loss of the curve the points were drawn from as references.

THE HIDDEN UNITS

```
w1 = net.stack[0].weight.detach().reshape(-1)    # (16, 1) -> (16,)   hidden weights
b1 = net.stack[0].bias.detach()                  # (16,)
v = net.stack[2].weight.detach().reshape(-1)     # (1, 16) -> (16,)   output weights
c = net.stack[2].bias.detach()                   # (1,)

grid = torch.linspace(-3, 3, 7)
units = torch.relu(grid[:, None] * w1 + b1) * v  # (7, 1) * (16,) -> (7, 16)
torch.allclose(units.sum(axis=1) + c, net(grid[:, None]).reshape(-1))   # -> True
```

Each hidden unit computes `relu(w * x + b)` of the single input: zero on one side of x = -b / w and a straight line on the other, a hinge. The output multiplies each hinge by its weight in `v`, adds them and adds `c`, which is what `torch.allclose` confirms on seven points: the network's prediction is a sum of 16 hinges, a broken line that bends wherever one of them does. Training moves the bends and changes the slopes. `.detach()` returns the parameters as tensors outside the recorded graph, since nothing here is differentiated.



Top: each hidden unit's share of the output over the range of the points. The 12 units drawn in blue bend inside the range, at the dotted lines; the 2 in grey are straight over the whole range, and the 2 units that are zero everywhere on it are not drawn. Bottom: their sum plus c, the network's prediction, a broken line that bends at the same places.

WITHOUT THE ACTIVATION

```
torch.manual_seed(0)
flat = nn.Sequential(nn.Linear(1, 16), nn.Linear(16, 1))   # no activation between the layers
opt = torch.optim.SGD(flat.parameters(), lr=0.05)
for step in range(3000):
    loss = loss_fn(flat(X), Y)
    opt.zero_grad()
    loss.backward()
    opt.step()

round(loss.item(), 4)   # -> 0.5256   the straight line's loss
```

Two linear layers in a row compute `W2 @ (W1 @ x + b1) + b2`, which is `(W2 @ W1) @ x + (W2 @ b1 + b2)`: one weight and one bias, a straight line again. With 16 units and 49 parameters this network ends at the line's loss, 0.5256. The bends come from the activation between the layers.

07

Regression on the register

`spend`, the amount a customer spent, predicted from `tenure_months`, `score` and `income`. The register marks every row as `train` or `test` in its `split` column: the models are trained on the training rows and measured on the test rows, which they do not see during training.

```
F = np.stack([tenure, score, income], axis=1)            # (9000, 3)
present = ~np.isnan(F).any(axis=1)                       # score and income have gaps
train = present & (split == "train")
test = present & (split == "test")
int(train.sum()), int(test.sum())                        # -> (5933, 1962)

mean, std = F[train].mean(axis=0), F[train].std(axis=0)  # from the training rows only
Xtr = torch.tensor((F[train] - mean) / std, dtype=torch.float32)
Xte = torch.tensor((F[test] - mean) / std, dtype=torch.float32)
ytr = torch.tensor(spend[train] / 1000, dtype=torch.float32).reshape(-1, 1)   # in thousands
yte = torch.tensor(spend[test] / 1000, dtype=torch.float32).reshape(-1, 1)
```

The features are standardised with the mean and the standard deviation of the training rows, and the test rows are scaled with the same two numbers, so nothing about the test rows reaches the model before it is measured. The target is divided by 1000, so its values lie around 1, the scale of the curve's targets.

```
def fit(model, X, Y, lr, steps):
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    for step in range(steps):
        loss = loss_fn(model(X), Y)
        opt.zero_grad()
        loss.backward()
        opt.step()

torch.manual_seed(0)
line = nn.Linear(3, 1)
fit(line, Xtr, ytr, lr=0.1, steps=200)
mlp = Net(3, 16)
fit(mlp, Xtr, ytr, lr=0.1, steps=1000)
```

RMSE AND MAE

```
def rmse(pred, target):
    return ((pred - target) ** 2).mean().sqrt().item() * 1000    # in units of spend

def mae(pred, target):
    return (pred - target).abs().mean().item() * 1000

with torch.no_grad():
    mean_pred = torch.full_like(yte, ytr.mean().item())   # the training mean on every row
    for name, pred in [("mean", mean_pred), ("line", line(Xte)), ("MLP", mlp(Xte))]:
        print(f"{name:<5} RMSE {rmse(pred, yte):6.1f}   MAE {mae(pred, yte):6.1f}")

# mean  RMSE  544.3   MAE  427.4
# line  RMSE  256.0   MAE  203.2
# MLP   RMSE  253.4   MAE  201.3
```

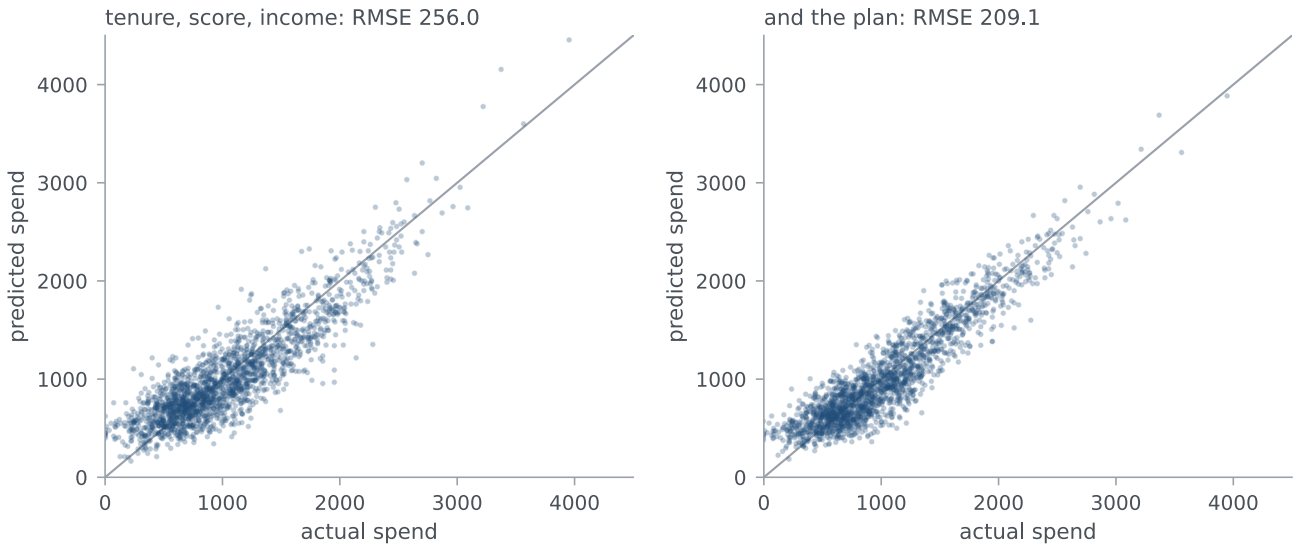
The error is reported in the unit of the target rather than its square. **Root mean squared error**, RMSE, is the square root of the mean squared error; **mean absolute error**, MAE, the mean of `abs(pred - target)`, is less affected by a few large errors. Either is quoted beside the error of predicting the mean of the training targets for every row, which for RMSE is close to the standard deviation of the target: 544.3 here, against 256.0 for the line. The MLP is 1 per cent better than the line, 253.4 against 256.0: on these three columns a hidden layer adds little to a straight line.

A FOURTH COLUMN

```
plans = ["basic", "standard", "premium", "legacy"]
onehot = np.stack([(plan == p).astype(np.float32) for p in plans], axis=1)   # (9000, 4)
Xtr4 = torch.cat([Xtr, torch.tensor(onehot[train])], axis=1)              # (5933, 7)
Xte4 = torch.cat([Xte, torch.tensor(onehot[test])], axis=1)

torch.manual_seed(0)
line4 = nn.Linear(7, 1)
fit(line4, Xtr4, ytr, lr=0.1, steps=500)
with torch.no_grad():
    round(rmse(line4(Xte4), yte), 1)       # -> 209.1
```

The plan, one-hot encoded, lowers the line's RMSE from 256.0 to 209.1, where the MLP on the three columns reached 253.4. `torch.cat` joins tensors along an existing axis, here the columns, like `np.concatenate`.



Predicted against actual spend for the 1962 test rows, one point per row, with the diagonal where the two are equal. Left: the line on tenure_months, score and income. Right: the same line with the plan added as four one-hot columns, whose points lie closer to the diagonal.

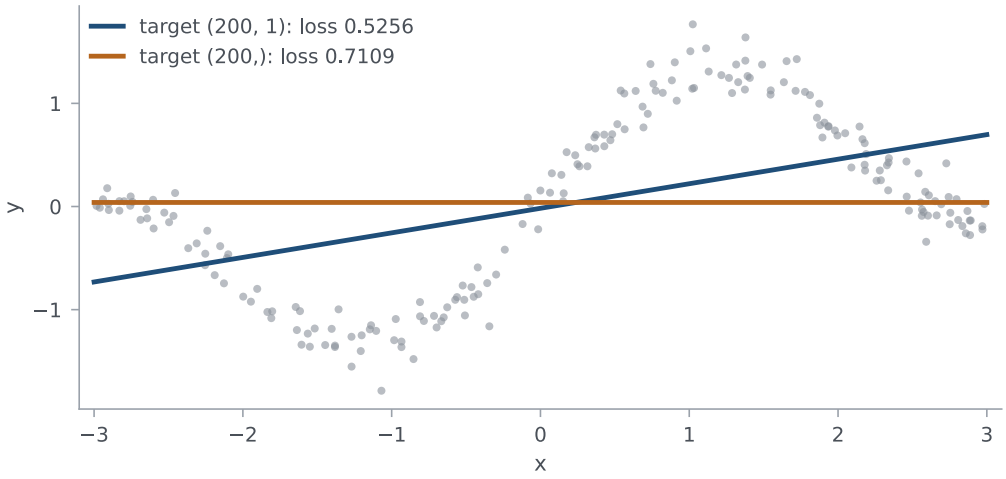
MISMATCHED TARGET SHAPE

```
nn.MSELoss()(torch.zeros(8, 1), torch.zeros(8))
# UserWarning: Using a target size (torch.Size([8])) that is different to the input size
# (torch.Size([8, 1])). This will likely lead to incorrect results due to broadcasting.
# Please ensure they have the same size.
```

A prediction of shape `(8, 1)` against a target of shape `(8,)` broadcasts to `(8, 8)` and averages 64 differences instead of 8. PyTorch only warns, and the loss is still computed and returned.

```
torch.manual_seed(0)
line = nn.Linear(1, 1)
opt = torch.optim.SGD(line.parameters(), lr=0.1)
for step in range(100):
    loss = loss_fn(line(X), Y.reshape(-1))  # prediction (200, 1), target (200,)
    opt.zero_grad()
    loss.backward()
    opt.step()

round(line.bias.item(), 4), round(loss.item(), 4)   # -> (0.0393, 0.7109)
round(y.mean(), 4), round(y.var(), 4)               # -> (0.0393, 0.7109)
```



The same line trained twice on the 200 points. With the target of shape (200, 1) it ends at the least-squares line; with the target of shape (200,) the loss averages over all 200 × 200 pairs of prediction and target, and the line ends flat at the mean of y, 0.0393.

With a target of shape `(200,)`, the loss averages the squared difference between every prediction and every one of the 200 targets, and that average is smallest when every prediction equals the mean of `y`. The weight goes to zero, the bias to 0.0393, the mean of `y`, and the loss stays at 0.7109, the variance of `y`: a flat line. Every target above has the shape `(n, 1)` of the predictions, which prevents it.

FURTHER READING

- torch.Tensor, for the dtypes and every tensor method.
- A Gentle Introduction to torch.autograd, and Autograd mechanics for leaves, `grad_fn` and no-grad mode.
- Build the Neural Network and Optimizing Model Parameters, the two PyTorch tutorials on `nn.Module` and the training loop.
- nn.Module, nn.Linear, nn.Sequential and nn.MSELoss.
- torch.optim, for the other optimizers and their arguments.